

Objectifs

Le but de cet atelier est la manipulation des deux méthodes de récupération et d'invocation des données.

Soit le diagramme suivant :

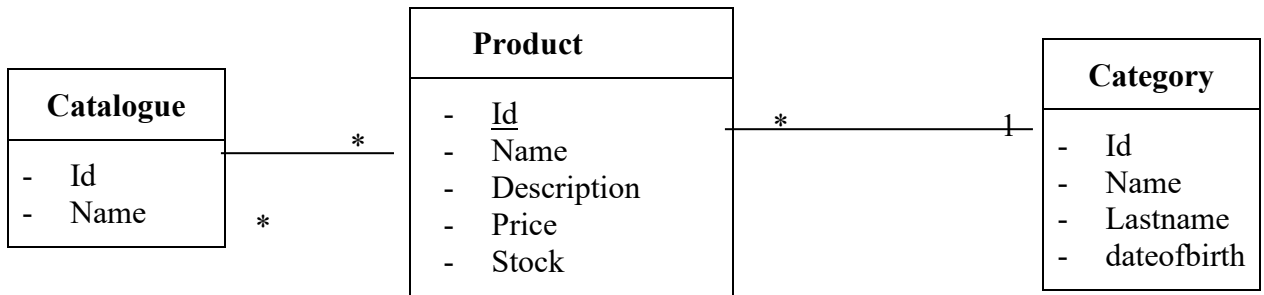


Figure 1: Diagramme de classe

Partie 1 : CRUD Produit

- 1- Créer la base de données « *atelier3* »
- 2- Créer une interface pour gérer les produits. La table suivante montre les détails :

Nom de la table	Product
id	Auto_increment
Name	String (20)
Description	String (200)
Price	Float
Stock	Integer

1. Créer le fichier de migration.
2. Générer la table « **Product** ».
3. Générer le modèle.
4. Générer le contrôleur de ressources associé à notre modèle « **Product** ».

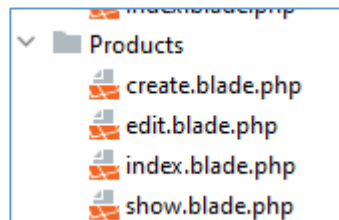
NB : Il faut maintenant remplir les différentes fonctions avec leurs traitements respectifs :

Nous avons comme correspondance :

- **index** : gère l'affichage de la page de listing des produits
- **create** : gère l'affichage de la page de création d'un produit
- **update** : gère la création d'un produit (suivant un formulaire de création par exemple)

- **show** : gère l'affichage de la page d'un produit
- **edit** : gère l'affichage de la page d'édition d'un produit
- **update** : gère la modification d'un produit (suivant un formulaire d'édition par exemple)
- **delete** : gère la suppression d'un produit.

5. Lister les routes de l'application.
6. Modifier le fichier *routes/web.php* en ajoutant la ligne permettant de générer les routes de ressources pour les produits.
7. Ré-lister les routes de l'application.
8. Créer les vues de notre application dans le dossier view/products :



Products						
ID	Nom	Description	Prix	Stock	Actions	
14	Miss Elza Gottlieb DDS	Autem sed beatae repudiandae autem. Minima quos quia dolor enim. Dolorem dolore et sed odio.	449	430	Détails	Delete
15	Ms. Alexandra Terry DDS	At nulla dolor dolorum ea est ea et. Sed quae saepe corrupti eaque enim consequatur.	687	221	Détails	Delete
20	Monserrat Little	Id sit amet expedita tempore. Cumque rem nihil eveniet amet et autem ut.	989	369	Détails	Delete

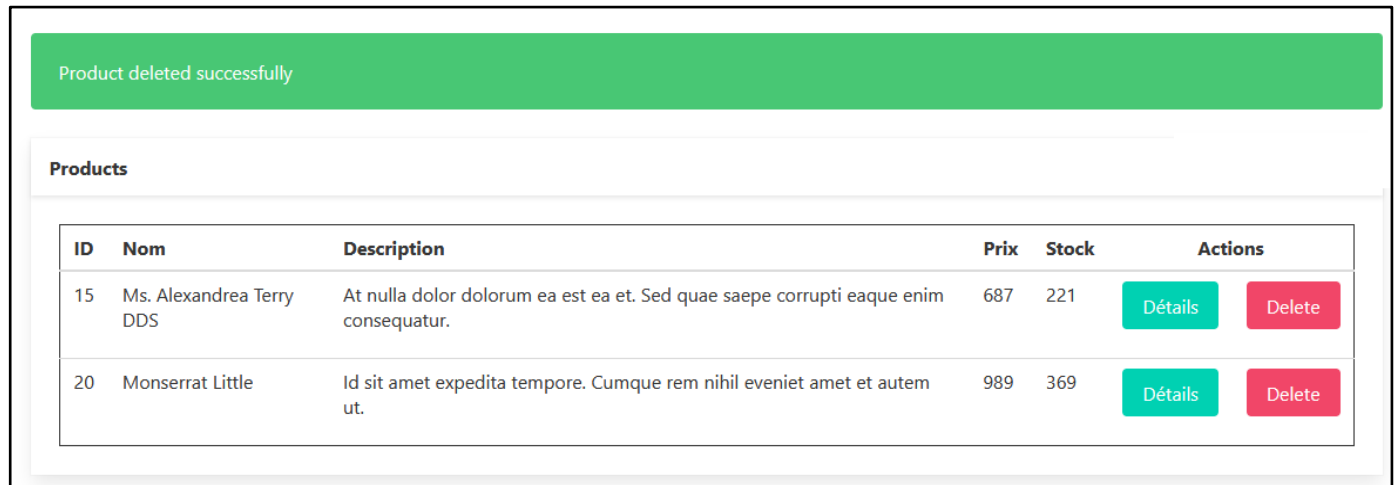


Figure 1: Page "index.blade.php"

Création d'un produit

Nom

Description

Prix

Stock

Ajouter

Figure 2: Page "create.blade.php"

9. Créer le fichier ProductFactory qui correspond au modèle « **product** » :

```
php artisan make:factory ProductFactory --model=Product
```

10. Modifier le fichier modèle en ajoutant le code suivant :

```
protected $fillable = [
    'name',
    'description',
    'price',
    'stock'
];
```

11. ajouter le code suivant au fichier ProductFactory :

```
return [
    'name' => $this->faker->name(20),
    'description' => $this->faker->text(200),
    'price' => $this->faker->randomNumber(3),
    'stock' => $this->faker->randomNumber(3),
];
```

12. Créer le fichier ProductSeeder qui correspond au modèle « **product** » :

php artisan make:seeder StudentSeeder

13. ajouter le code suivant au fichier ProductSeeder :

```
public function run()
{
    Student::factory(10)->create();
}
```

14. Ajouter le code suivant au fichier DatabaseSeeder :

```
public function run()
{
    $this->call([ProductSeeder::class]);
}
```

15. Exécuter maintenant la commande :

php artisan db:seed

16. Vérifier que les données ont été ajoutées à la base de données.

Partie 2 : CRUD Category

1. Créer une migration pour ajouter la colonne de la clé étrangère pour l'association entre **Product** et **Category**. La table suivante montre les détails d'une catégorie :

Nom de la table	Category
id	Auto increment
Name	String (20)

2. Générer le modèle.
3. Créer l'association ManyToOne entre Product et Category dans les modèles correspondants.

4. Générer le contrôleur de ressources associé au modèle « *Category* ».
5. Modifier l'interface d'ajout de produit avec une liste déroulante pour choisir la catégorie comme indiqué dans la figure ci-dessous :

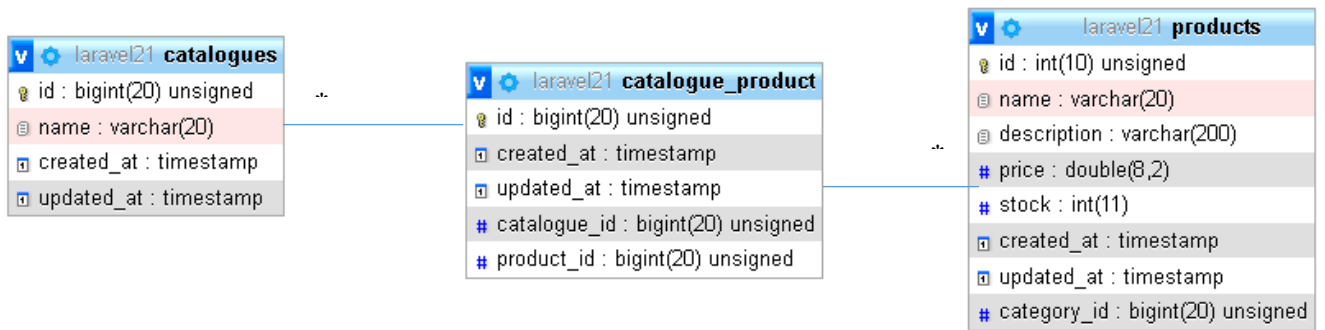
The image shows a web form titled "Création d'un produit". It contains the following elements:

- Nom**: A text input field.
- Description**: A large text area.
- Prix**: A text input field with a small downward arrow on the right.
- Stock**: A text input field with a small downward arrow on the right.
- Categories**: A dropdown menu showing "catg1" with a blue downward arrow. This entire section is highlighted with a red rectangular border.
- Ajouter**: A blue button at the bottom left.

Figure 3: Page "create.blade.php"

Partie 3: Relation Many to Many

1. Générer le modèle Catalogue ainsi que son fichier de migration
2. Générer le fichier de migration de la table **pivot** avec le nom :
`create_catalogue_product_table`



3. Ajouter le code nécessaire pour la table pivot.
4. Modifier les entités Product et Catalogues pour assurer la relation Many to Many
5. Modifier l'interface d'ajout d'un film de façon qu'on puisse choisir plus qu'un catalogue dans la liste.

Création d'un produit

Nom

Description

Prix

Stock

Categories

Catalogue 1
 Catalogue 2

6. Modifier le code de la méthode store.
7. Modifier le code de la méthode update.

Partie 4 : Validation des données

1. Ajouter le code nécessaire pour que les champs du formulaire « Ajout produit » respectent les règles suivantes :
 - Tous les champs sont obligatoires

- Le champ nom est de taille maximale 2 caractères
- Le champ description est de taille maximale 100 caractères
- Le prix et le stock sont numériques

2. Afficher les erreurs de validation

Nom de produit

The name must not be greater than 20 characters.